

ESTRUTURA DE DADOS NÃO-LINEARES

Implementação de Árvore AVL com Herança de ABP

Autora: Larissa Samara | Linguagem: Java | Complexidade: $O(\log n)$

1. Fundamentação Teórica e Fórmulas de Recálculo

A Árvore AVL é uma árvore binária de pesquisa autobalanceada em que a diferença entre as alturas das subárvores esquerda e direita de qualquer nó (denominada Fator de Balanceamento - FB) é de no máximo 1 em valor absoluto. Isso garante que a altura máxima da árvore seja $h \leq 1,44 * \log_2(n)$, conferindo complexidade estritamente $O(\log n)$ para as operações básicas de busca, inclusão e remoção.

Definição do Fator de Balanceamento: $FB(u) = \text{altura}(\text{subárvore esquerda}) - \text{altura}(\text{subárvore direita})$. Portanto, valores positivos indicam que a esquerda é mais alta (+1), zero indica equilíbrio perfeito (0), e valores negativos indicam que a direita é mais alta (-1). Desbalanceamentos ocorrem quando FB atinge +2 ou -2.

Conforme solicitado na especificação da disciplina, para garantir que as rotações executem estritamente em tempo constante $O(1)$ sem necessidade de recalcular alturas a partir das folhas, foram empregadas as fórmulas deduzidas em aula:

Fórmulas para Rotação Simples para a Esquerda (S.E.): $FB'(A) = FB(A) + 1 - \min(FB(B), 0)$
 $FB'(B) = FB(B) + 1 + \max(FB'(A), 0)$

Fórmulas para Rotação Simples para a Direita (S.D.): $FB'(A) = FB(A) - 1 - \max(FB(B), 0)$
 $FB'(B) = FB(B) - 1 + \min(FB'(A), 0)$

2. Detalhamento Passo a Passo do Exemplo da Especificação

2.1 Inserções Iniciais e Inserção do Nó 25 (Rotação S.E. em 15)

Inicialmente, inserem-se as chaves 10, 5, 15, 2, 8, 22. A árvore resultante apresenta equilíbrio em todos os nós, com exceção do nó 15 que possui $FB = -1$ devido ao seu filho direito 22.

Propagação do desbalanceamento: Ao inserir a chave 25, ela é posicionada como filha à direita do nó 22. No retorno da recursão (backtracking):

- O nó 22 tem sua subárvore direita acrescida de altura: seu FB passa de 0 para -1. - O nó 15, que já possuía $FB = -1$, recebe a propagação do aumento de altura da subárvore direita: seu FB atinge -2 (desbalanceamento detectado à direita). - Como os sinais de $FB(15) = -2$ e $FB(22) = -1$ são concordantes (ambos negativos), aplica-se a Rotação Simples para a Esquerda (S.E.) com pivô no nó 15.

Aplicação das Fórmulas de Recálculo em S.E.: $FB'(15) = -2 + 1 - \min(-1, 0) = -2 + 1 - (-1) = 0$
 $FB'(22) = -1 + 1 + \max(0, 0) = 0$ Resultado: O nó 22 torna-se a nova raiz da subárvore com 15 à esquerda e 25 à direita, ambos com $FB = 0$.

2.2 Remoção do Nó 5 e Substituição pelo Sucessor

Etapas da remoção: O nó 5 possui dois filhos (2 à esquerda e 8 à direita). Pela regra clássica de

ABP/AVL com 2 filhos, o nó é substituído pelo seu sucessor in-order (o menor elemento da sua subárvore direita, que neste caso é a folha 8):

1. A chave 8 substitui o valor 5 no nó de destino. 2. O nó original 8 (que era folha) é removido da subárvore direita. 3. Ao remover da subárvore direita do nó 8 recém-alocado, a altura da direita diminui de 1 para 0. 4. Como a subárvore esquerda possui o nó 2 (altura 1), o fator de balanceamento é recalculado: $FB(8) = h(esq) - h(dir) = 1 - 0 = +1$. 5. A altura total da subárvore enraizada em 8 permaneceu 2 ($\max(1, 0) + 1 = 2$), a mesma que tinha antes da remoção. Consequentemente, a variação de altura NÃO se propaga para a raiz 10, mantendo $FB(10) = 0$.

3. Código-Fonte Completo (Java)

O código foi desenvolvido em conformidade com as boas práticas de orientação a objetos, utilizando herança direta de ABP e tipagem estrita. Os blocos abaixo utilizam formatação monoespçada (Consolas 9.5pt) com realce sintático no padrão da IDE Eclipse (palavras-chave em roxo/negrito, comentários em verde itálico, strings em azul).

Classe NoABP.java

```
/**
 * Classe que representa um nó na Árvore Binária de Pesquisa (ABP).
 * Serve como classe base para os nós da Árvore AVL através de herança.
 *
 * @author Larissa Samara
 */
public class NoABP {
    public int chave;
    public NoABP esquerdo;
    public NoABP direito;

    /**
     * Construtor do nó básico de ABP.
     * @param chave Valor inteiro armazenado no nó.
     */
    public NoABP(int chave) {
        this.chave = chave;
        this.esquerdo = null;
        this.direito = null;
    }
}
```

Classe ArvoreBinariaPesquisa.java

```
import java.util.ArrayList;
import java.util.Comparator;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

/**
 * Implementação da Árvore Binária de Pesquisa (ABP).
 * Fornece as operações fundamentais (busca, inserção, remoção, exibição)
 * e serve como superclasse para a ArvoreAVL.
 *
 * @author Larissa Samara
 */
public class ArvoreBinariaPesquisa {
    protected NoABP raiz;

    public ArvoreBinariaPesquisa() {
        this.raiz = null;
    }
}
```

```

public NoABP getRaiz() {
    return this.raiz;
}

public boolean estaVazia() {
    return this.raiz == null;
}

/**
 * Busca por uma chave na árvore.
 * Complexidade de tempo:  $O(h)$ , sendo  $O(\log n)$  se balanceada.
 * @param chave Valor a ser buscado.
 * @return true se a chave existe na árvore, false caso contrário.
 */
public boolean buscar(int chave) {
    return buscarRec(this.raiz, chave) != null;
}

protected NoABP buscarRec(NoABP atual, int chave) {
    if (atual == null || atual.chave == chave) {
        return atual;
    }
    if (chave < atual.chave) {
        return buscarRec(atual.esquerdo, chave);
    } else {
        return buscarRec(atual.direito, chave);
    }
}

/**
 * Inserção padrão de ABP.
 * @param chave Valor a ser inserido.
 */
public void inserir(int chave) {
    this.raiz = inserirRec(this.raiz, chave);
}

protected NoABP inserirRec(NoABP atual, int chave) {
    if (atual == null) {
        return new NoABP(chave);
    }
    if (chave < atual.chave) {
        atual.esquerdo = inserirRec(atual.esquerdo, chave);
    } else if (chave > atual.chave) {
        atual.direito = inserirRec(atual.direito, chave);
    }
    return atual;
}

/**
 * Remoção padrão de ABP.
 * @param chave Valor a ser removido.
 */
public void remover(int chave) {
    this.raiz = removerRec(this.raiz, chave);
}

protected NoABP removerRec(NoABP atual, int chave) {
    if (atual == null) {
        return null;
    }

    if (chave < atual.chave) {

```

```

        atual.esquerdo = removerRec(atual.esquerdo, chave);
    } else if (chave > atual.chave) {
        atual.direito = removerRec(atual.direito, chave);
    } else {
        // Caso 1 e Caso 2: nó folha ou com apenas 1 filho
        if (atual.esquerdo == null) {
            return atual.direito;
        } else if (atual.direito == null) {
            return atual.esquerdo;
        }

        // Caso 3: nó com 2 filhos
        // Substitui pelo sucessor (menor elemento da subárvore direita)
        NoABP sucessor = obterMinimo(atual.direito);
        atual.chave = sucessor.chave;
        atual.direito = removerRec(atual.direito, sucessor.chave);
    }
    return atual;
}

/**
 * Encontra o nó com menor valor na subárvore.
 */
protected NoABP obterMinimo(NoABP atual) {
    NoABP temp = atual;
    while (temp != null && temp.esquerdo != null) {
        temp = temp.esquerdo;
    }
    return temp;
}

/**
 * Exibe a árvore visualmente por níveis, alinhando as colunas
 * de acordo com a ordem simétrica (in-order).
 */
public void mostrar() {
    mostrarFormatado(this.raiz);
}

protected void mostrarFormatado(NoABP raizSubarvore) {
    if (raizSubarvore == null) {
        System.out.println(" (Árvore vazia)");
        return;
    }

    Map<NoABP, Integer> xPos = new HashMap<>();
    Map<NoABP, Integer> yPos = new HashMap<>();
    int[] col = new int[]{0};
    calcularPosicoes(raizSubarvore, 0, col, xPos, yPos);

    int profundidadeMaxima = 0;
    for (int d : yPos.values()) {
        if (d > profundidadeMaxima) {
            profundidadeMaxima = d;
        }
    }

    int larguraColuna = 10;
    for (int d = 0; d <= profundidadeMaxima; d++) {
        StringBuilder linha = new StringBuilder();
        List<NoABP> nosDoNivel = new ArrayList<>();
        for (Map.Entry<NoABP, Integer> entry : yPos.entrySet()) {
            if (entry.getValue() == d) {

```

```

        nosDoNivel.add(entry.getKey());
    }
}
nosDoNivel.sort(Comparator.comparingInt(xPos::get));

for (NoABP n : nosDoNivel) {
    int colunaAlvo = xPos.get(n) * larguraColuna + 4;
    while (linha.length() < colunaAlvo) {
        linha.append(" ");
    }
    String rotulo;
    if (n instanceof NoAVL) {
        rotulo = n.chave + "[" + ((NoAVL) n).fb + "]";
    } else {
        rotulo = String.valueOf(n.chave);
    }
    linha.append(rotulo);
}
System.out.println(linha.toString());
}
}

private void calcularPosicoes(NoABP n, int profundidade, int[] col, Map<NoABP, Integer> xPos, Map<NoABP, Integer> yPos) {
    if (n == null) return;
    calcularPosicoes(n.esquerdo, profundidade + 1, col, xPos, yPos);
    xPos.put(n, col[0]++);
    yPos.put(n, profundidade);
    calcularPosicoes(n.direito, profundidade + 1, col, xPos, yPos);
}
}

```

Classe NoAVL.java

```

/**
 * Classe que representa um nó na Árvore AVL, estendendo NoABP por herança.
 * Adiciona o atributo de Fator de Balanceamento (FB).
 *
 * Convenção adotada na disciplina:
 * FB(nó) = altura(subárvore esquerda) - altura(subárvore direita)
 *
 * Valores possíveis para nós balanceados em AVL: -1, 0, +1.
 * Desbalanceamento ocorre quando FB atinge +2 ou -2.
 *
 * @author Larissa Samara
 */

```

```

public class NoAVL extends NoABP {
    public int fb;

```

```

    /**
     * Construtor do nó AVL.
     * @param chave Valor inteiro a ser armazenado.
     */
    public NoAVL(int chave) {
        super(chave);
        this.fb = 0;
    }
}

```

Classe ArvoreAVL.java

```

/**
 * Implementação da Árvore AVL através de herança de ArvoreBinariaPesquisa (ABP).
 *
 * Todas as operações básicas (busca, inserção, remoção) rodam estritamente em O(log n).
 * O recálculo dos fatores de balanceamento após as rotações é feito em O(1) utilizando

```

* as fórmulas matemáticas deduzidas em aula.
 *
 * Convenção do Fator de Balanceamento (FB):
 * $FB(p) = \text{altura}(\text{subárvore esquerda}) - \text{altura}(\text{subárvore direita})$
 *
 * Fórmulas de Atualização de FB após Rotação Simples:
 * 1. Rotação Simples para a Esquerda (S.E.):
 * $FB'(A) = FB(A) + 1 - \min(FB(B), 0)$
 * $FB'(B) = FB(B) + 1 + \max(FB'(A), 0)$
 *
 * 2. Rotação Simples para a Direita (S.D.):
 * $FB'(A) = FB(A) - 1 - \max(FB(B), 0)$
 * $FB'(B) = FB(B) - 1 + \min(FB'(A), 0)$
 *
 * @author Larissa Samara
 */

```
public class ArvoreAVL extends ArvoreBinariaPesquisa {

    // Flag que indica se a altura da subárvore variou durante a recursão
    private boolean mudouAltura;

    public ArvoreAVL() {
        super();
    }

    /**
     * Inserção com balanceamento AVL em  $O(\log n)$ .
     * @param chave Valor a ser inserido.
     */
    @Override
    public void inserir(int chave) {
        this.mudouAltura = false;
        this.raiz = inserirAVL((NoAVL) this.raiz, chave);
    }

    private NoAVL inserirAVL(NoAVL atual, int chave) {
        if (atual == null) {
            this.mudouAltura = true;
            return new NoAVL(chave);
        }

        if (chave < atual.chave) {
            atual.esquerdo = inserirAVL((NoAVL) atual.esquerdo, chave);
            if (this.mudouAltura) {
                switch (atual.fb) {
                    case -1:
                        // Tinha subárvore direita mais alta; inseriu na esquerda -> agora equilibrado
                        atual.fb = 0;
                        this.mudouAltura = false;
                        break;
                    case 0:
                        // Estava equilibrado; inseriu na esquerda -> subárvore esquerda ficou mais alta
                        atual.fb = 1;
                        this.mudouAltura = true;
                        break;
                    case 1:
                        // Já estava +1; inseriu na esquerda -> FB iria para +2 (desbalanceamento)
                        atual = balancearEsquerda(atual);
                        this.mudouAltura = false;
                        break;
                }
            }
        }
        else if (chave > atual.chave) {
```

```

    atual.direito = inserirAVL((NoAVL) atual.direito, chave);
    if (this.mudouAltura) {
        switch (atual.fb) {
            case 1:
                // Tinha subárvore esquerda mais alta; inseriu na direita -> agora equilibrado
                atual.fb = 0;
                this.mudouAltura = false;
                break;
            case 0:
                // Estava equilibrado; inseriu na direita -> subárvore direita ficou mais alta
                atual.fb = -1;
                this.mudouAltura = true;
                break;
            case -1:
                // Já estava -1; inseriu na direita -> FB iria para -2 (desbalanceamento)
                atual = balancearDireita(atual);
                this.mudouAltura = false;
                break;
        }
    }
} else {
    // Chave duplicada: não permite duplicidade
    this.mudouAltura = false;
}

return atual;
}

/**
 * Trata o desbalanceamento após inserção na subárvore esquerda (FB iria para +2).
 */
private NoAVL balancearEsquerda(NoAVL pivo) {
    NoAVL filhoEsq = (NoAVL) pivo.esquerdo;
    if (filhoEsq.fb >= 0) {
        // Sinais iguais (+2 e +1): Rotação Simples à Direita (S.D.)
        return rotacaoSimplesDireita(pivo);
    } else {
        // Sinais opostos (+2 e -1): Rotação Dupla à Direita (D.D.)
        return rotacaoDuplaDireita(pivo);
    }
}

/**
 * Trata o desbalanceamento após inserção na subárvore direita (FB iria para -2).
 */
private NoAVL balancearDireita(NoAVL pivo) {
    NoAVL filhoDir = (NoAVL) pivo.direito;
    if (filhoDir.fb <= 0) {
        // Sinais iguais (-2 e -1): Rotação Simples à Esquerda (S.E.)
        return rotacaoSimplesEsquerda(pivo);
    } else {
        // Sinais opostos (-2 e +1): Rotação Dupla à Esquerda (D.E.)
        return rotacaoDuplaEsquerda(pivo);
    }
}

/**
 * Rotação Simples para a Esquerda (S.E.).
 *
 * Estrutura:
 *   A         B
 *   \        /\
 *   B  ==>  A  T3

```

```

*      /\      \
*      T2 T3      T2
*
* Fórmulas de recálculo:
*  $FB'(A) = FB(A) + 1 - \min(FB(B), 0)$ 
*  $FB'(B) = FB(B) + 1 + \max(FB'(A), 0)$ 
*
* @param A Nó desbalanceado (pivô)
* @return Nova raiz da subárvore (B)
*/
public NoAVL rotacaoSimplesEsquerda(NoAVL A) {
    NoAVL B = (NoAVL) A.direito;
    A.direito = B.esquerdo;
    B.esquerdo = A;

    // Se A.fb estava em -1 no momento que detectou desbalanceamento na inserção,
    // o valor efetivo antes da rotação é -2.
    int fbA_antes = (A.fb == -1) ? -2 : A.fb;

    int novoFbA = fbA_antes + 1 - Math.min(B.fb, 0);
    int novoFbB = B.fb + 1 + Math.max(novoFbA, 0);

    A.fb = novoFbA;
    B.fb = novoFbB;

    return B;
}

/**
* Rotação Simples para a Direita (S.D.).
*
* Estrutura:
*      A          B
*      /\      /\
*      B  ==>  T1 A
*      /\      /
*      T1 T2      T2
*
* Fórmulas de recálculo:
*  $FB'(A) = FB(A) - 1 - \max(FB(B), 0)$ 
*  $FB'(B) = FB(B) - 1 + \min(FB'(A), 0)$ 
*
* @param A Nó desbalanceado (pivô)
* @return Nova raiz da subárvore (B)
*/
public NoAVL rotacaoSimplesDireita(NoAVL A) {
    NoAVL B = (NoAVL) A.esquerdo;
    A.esquerdo = B.direito;
    B.direito = A;

    // Se A.fb estava em +1 no momento que detectou desbalanceamento na inserção,
    // o valor efetivo antes da rotação é +2.
    int fbA_antes = (A.fb == 1) ? 2 : A.fb;

    int novoFbA = fbA_antes - 1 - Math.max(B.fb, 0);
    int novoFbB = B.fb - 1 + Math.min(novoFbA, 0);

    A.fb = novoFbA;
    B.fb = novoFbB;

    return B;
}

```

```

/**
 * Rotação Dupla para a Esquerda (D.E. ou Direita-Esquerda).
 * Primeiro executa uma rotação simples à direita no filho direito,
 * e em seguida uma rotação simples à esquerda no pai.
 *
 * @param A Nó desbalanceado
 * @return Nova raiz da subárvore
 */
public NoAVL rotacaoDuplaEsquerda(NoAVL A) {
    A.direito = rotacaoSimplesDireita((NoAVL) A.direito);
    return rotacaoSimplesEsquerda(A);
}

/**
 * Rotação Dupla para a Direita (D.D. ou Esquerda-Direita).
 * Primeiro executa uma rotação simples à esquerda no filho esquerdo,
 * e em seguida uma rotação simples à direita no pai.
 *
 * @param A Nó desbalanceado
 * @return Nova raiz da subárvore
 */
public NoAVL rotacaoDuplaDireita(NoAVL A) {
    A.esquerdo = rotacaoSimplesEsquerda((NoAVL) A.esquerdo);
    return rotacaoSimplesDireita(A);
}

/**
 * Remoção com balanceamento AVL em  $O(\log n)$ .
 * @param chave Valor a ser removido.
 */
@Override
public void remover(int chave) {
    this.mudouAltura = false;
    this.raiz = removerAVL((NoAVL) this.raiz, chave);
}

private NoAVL removerAVL(NoAVL atual, int chave) {
    if (atual == null) {
        this.mudouAltura = false;
        return null;
    }

    if (chave < atual.chave) {
        atual.esquerdo = removerAVL((NoAVL) atual.esquerdo, chave);
        if (this.mudouAltura) {
            atual = aposRemocaoEsquerda(atual);
        }
    } else if (chave > atual.chave) {
        atual.direito = removerAVL((NoAVL) atual.direito, chave);
        if (this.mudouAltura) {
            atual = aposRemocaoDireita(atual);
        }
    } else {
        // Encontrou o nó a remover
        if (atual.esquerdo == null) {
            this.mudouAltura = true;
            return (NoAVL) atual.direito;
        } else if (atual.direito == null) {
            this.mudouAltura = true;
            return (NoAVL) atual.esquerdo;
        } else {
            // Nó com 2 filhos: substitui pelo sucessor (menor da subárvore direita)
            NoAVL sucessor = (NoAVL) obterMinimo(atual.direito);

```

```

        atual.chave = sucessor.chave;
        atual.direito = removerAVL((NoAVL) atual.direito, sucessor.chave);
        if (this.mudouAltura) {
            atual = aposRemocaoDireita(atual);
        }
    }
}
return atual;
}

/**
 * Trata o ajuste de FB e eventuais rotações quando a subárvore esquerda diminui de altura.
 */
private NoAVL aposRemocaoEsquerda(NoAVL atual) {
    switch (atual.fb) {
        case 1:
            // Tinha esquerda mais alta (+1); a esquerda diminuiu -> agora equilibrado (0)
            atual.fb = 0;
            this.mudouAltura = true;
            break;
        case 0:
            // Estava equilibrado (0); a esquerda diminuiu -> subárvore direita ficou mais alta (-1)
            atual.fb = -1;
            this.mudouAltura = false; // Altura máxima da subárvore não diminuiu
            break;
        case -1:
            // Já tinha subárvore direita mais alta (-1); esquerda diminuiu -> desbalanceou (iria para -2)
            atual.fb = -2;
            NoAVL dir = (NoAVL) atual.direito;
            if (dir.fb <= 0) {
                // Rotação Simples à Esquerda (S.E.)
                atual = rotacaoSimplesEsquerda(atual);
                this.mudouAltura = (atual.fb == 0);
            } else {
                // Rotação Dupla à Esquerda (D.E.)
                atual = rotacaoDuplaEsquerda(atual);
                this.mudouAltura = true;
            }
            break;
    }
    return atual;
}

/**
 * Trata o ajuste de FB e eventuais rotações quando a subárvore direita diminui de altura.
 */
private NoAVL aposRemocaoDireita(NoAVL atual) {
    switch (atual.fb) {
        case -1:
            // Tinha direita mais alta (-1); a direita diminuiu -> agora equilibrado (0)
            atual.fb = 0;
            this.mudouAltura = true;
            break;
        case 0:
            // Estava equilibrado (0); a direita diminuiu -> subárvore esquerda ficou mais alta (+1)
            atual.fb = 1;
            this.mudouAltura = false; // Altura máxima da subárvore não diminuiu
            break;
        case 1:
            // Já tinha subárvore esquerda mais alta (+1); direita diminuiu -> desbalanceou (iria para +2)
            atual.fb = 2;
            NoAVL esq = (NoAVL) atual.esquerdo;
            if (esq.fb >= 0) {

```

```

        // Rotação Simples à Direita (S.D.)
        atual = rotacaoSimplesDireita(atual);
        this.mudouAltura = (atual.fb == 0);
    } else {
        // Rotação Dupla à Direita (D.D.)
        atual = rotacaoDuplaDireita(atual);
        this.mudouAltura = true;
    }
    break;
}
return atual;
}
}

```

Classe Main.java

```
import java.util.Scanner;
```

```
/**
```

```
 * Classe principal de teste para a Árvore AVL.
```

```
 *
```

```
 * Permite ao usuário interagir via terminal realizando:
```

```
 * - Inclusão de nós
```

```
 * - Remoção de nós
```

```
 * - Busca de nós
```

```
 * - Visualização da árvore com chaves e Fatores de Balanceamento [FB]
```

```
 * - Execução automatizada do caso de teste da aula com explicação passo a passo
```

```
 *
```

```
 * @author Larissa Samara
```

```
 */
```

```
public class Main {
```

```

    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        ArvoreAVL avl = new ArvoreAVL();
        int opcao = -1;

```

```

System.out.println("=====
");

```

```

    System.out.println("          SISTEMA DE TESTES - ÁRVORE AVL (HERANÇA DE ABP)          ");
    System.out.println("          Aluna: Larissa Samara          ");

```

```

System.out.println("=====
");

```

```

    while (opcao != 0) {
        System.out.println("\n----- MENU -----");
        System.out.println(" 1 - Inserir nó");
        System.out.println(" 2 - Remover nó");
        System.out.println(" 3 - Buscar nó");
        System.out.println(" 4 - Mostrar árvore (com chaves e FB)");
        System.out.println(" 5 - Executar exemplo completo da aula (passo a passo)");
        System.out.println(" 6 - Limpar / Reiniciar árvore");
        System.out.println(" 0 - Sair");
        System.out.print("Escolha uma opção: ");

```

```

        try {
            String entrada = scanner.nextLine().trim();
            if (entrada.isEmpty()) continue;
            opcao = Integer.parseInt(entrada);
        } catch (NumberFormatException e) {
            System.out.println(">> Opção inválida! Digite um número inteiro.");
            continue;
        }
    }
}

```

```

switch (opcao) {
    case 1:
        System.out.print("Digite o valor inteiro para inserir: ");
        try {
            int valor = Integer.parseInt(scanner.nextLine().trim());
            if (avl.buscar(valor)) {
                System.out.println(">> A chave " + valor + " já existe na árvore (duplicatas não permitidas).");
            } else {
                avl.inserir(valor);
                System.out.println(">> Valor " + valor + " inserido com sucesso!");
                System.out.println("\nÁrvore atual:");
                avl.mostrar();
            }
        } catch (NumberFormatException e) {
            System.out.println(">> Valor inválido! Digite um número inteiro.");
        }
        break;

    case 2:
        System.out.print("Digite o valor inteiro para remover: ");
        try {
            int valor = Integer.parseInt(scanner.nextLine().trim());
            if (!avl.buscar(valor)) {
                System.out.println(">> A chave " + valor + " não foi encontrada na árvore.");
            } else {
                avl.remover(valor);
                System.out.println(">> Valor " + valor + " removido com sucesso!");
                System.out.println("\nÁrvore atual:");
                avl.mostrar();
            }
        } catch (NumberFormatException e) {
            System.out.println(">> Valor inválido! Digite um número inteiro.");
        }
        break;

    case 3:
        System.out.print("Digite o valor inteiro para buscar: ");
        try {
            int valor = Integer.parseInt(scanner.nextLine().trim());
            boolean achou = avl.buscar(valor);
            if (achou) {
                System.out.println(">> CHAVE ENCONTRADA: " + valor + " está presente na árvore.");
            } else {
                System.out.println(">> CHAVE NÃO ENCONTRADA: " + valor + " não existe na árvore.");
            }
        } catch (NumberFormatException e) {
            System.out.println(">> Valor inválido! Digite um número inteiro.");
        }
        break;

    case 4:
        System.out.println("\nVisualização da Árvore AVL (Chave [FB]):");
        avl.mostrar();
        break;

    case 5:
        executarExemploAula(scanner);
        // Atualiza a árvore em memória com o exemplo da aula
        avl = new ArvoreAVL();
        int[] nos = {10, 8, 15, 2, 22, 25}; // Estado final após remoção do 5
        for (int n : nos) avl.inserir(n);
        break;
}

```

```

        case 6:
            avl = new ArvoreAVL();
            System.out.println(">> Árvore reiniciada com sucesso! (Vazia)");
            break;

        case 0:
            System.out.println(">> Encerrando o programa. Até logo!");
            break;

        default:
            System.out.println(">> Opção inválida. Escolha entre 0 e 6.");
    }
}

scanner.close();
}

/**
 * Executa passo a passo a demonstração exigida na especificação do trabalho:
 * 1. Inserção de 10, 5, 15, 2, 8, 22.
 * 2. Inserção de 25 com rotação S.E. em 15.
 * 3. Remoção de 5 com substituição por sucessor (8).
 */
private static void executarExemploAula(Scanner scanner) {

System.out.println("\n=====
==");
    System.out.println("          DEMONSTRAÇÃO DO EXEMPLO DA ESPECIFICAÇÃO DA AULA          ");

System.out.println("=====
==");

    ArvoreAVL arvoreExemplo = new ArvoreAVL();
    int[] chavesIniciais = {10, 5, 15, 2, 8, 22};

    System.out.println("\n[PASSO 1] Inserindo chaves iniciais: 10, 5, 15, 2, 8, 22...");
    for (int c : chavesIniciais) {
        arvoreExemplo.inserir(c);
    }

    System.out.println("\nÁrvore resultante após inserções iniciais:");
    arvoreExemplo.mostrar();

    System.out.println("\n[PASSO 2] Inserindo chave 25...");
    System.out.println("Ao inserir 25 na subárvore direita de 22 (que é filho de 15):");
    System.out.println(" - 22 fica com FB = -1");
    System.out.println(" - 15 fica com FB = -2 (DESBALANCEAMENTO detectado!)");
    System.out.println("\nEstado conceitual antes da rotação:");
    System.out.println("                10[0]");
    System.out.println("        5[0]                15[-2]");
    System.out.println("    2[0]                8[0]                22[-1]");
    System.out.println("                25[0]");

    System.out.println("\nComo FB(15) = -2 e FB(22) = -1 (sinais iguais negativos):");
    System.out.println(">> Rotação Simples para a Esquerda (S.E.) aplicada no nó 15.");
    System.out.println("Aplicação das fórmulas de recálculo:");
    System.out.println(" FB'(15) = FB(15) + 1 - min(FB(22), 0) = -2 + 1 - (-1) = 0");
    System.out.println(" FB'(22) = FB(22) + 1 + max(FB'(15), 0) = -1 + 1 + 0 = 0");

    arvoreExemplo.inserir(25);
    System.out.println("\nÁrvore após rotação S.E. em 15:");
    arvoreExemplo.mostrar();

```

```

System.out.println("\n[PASSO 3] Removendo chave 5...");
System.out.println("Detalhe da implementação:");
System.out.println(" 1. O nó 5 possui dois filhos (2 e 8).");
System.out.println(" 2. Localiza o sucessor in-order: o menor nó da subárvore direita, que é o 8.");
System.out.println(" 3. Copia a chave 8 para a posição do 5.");
System.out.println(" 4. Remove o nó folha original 8 da subárvore direita.");
System.out.println(" 5. A subárvore direita de 8 diminuiu de altura. Recalcula FB(8):");
System.out.println("    FB(8) = altura(esq=2) - altura(dir=vazio) = 1 - 0 = +1.");
System.out.println(" 6. Como a altura total da subárvore enraizada em 8 permaneceu 2,");
System.out.println("    a alteração de altura NÃO se propaga para a raiz 10 (FB(10) continua 0).");

arvoreExemplo.remove(5);
System.out.println("\nÁrvore resultante após remover 5:");
arvoreExemplo.mostrar();

System.out.println("\n=====
==");
System.out.println(">> Demonstração concluída. A árvore atual do menu agora reflete este estado.");
}
}

```

4. Registro de Execução dos Testes no Terminal

```

=====
SISTEMA DE TESTES - RVORE AVL (HERANÇA DE ABP)
Aluna: Larissa Samara
=====

[PASSO 1] Inserindo chaves iniciais: 10, 5, 15, 2, 8, 22...

rvore resultante após inserções iniciais:
      10[0]
     /  \
   5[0]  15[-1]
  /  \    \
2[0] 8[0] 22[0]

[PASSO 2] Inserindo chave 25...
Ao inserir 25 na subárvore direita de 22 (que filho de 15):
- 22 fica com FB = -1
- 15 fica com FB = -2 (DESBALANCEAMENTO detectado!)

Como FB(15) = -2 e FB(22) = -1 (sinais iguais negativos):
>> Rotação Simples para a Esquerda (S.E.) aplicada no nó 15.
Aplicação das fórmulas de recálculo:
FB(15) = FB(15) + 1 - min(FB(22), 0) = -2 + 1 - (-1) = 0
FB(22) = FB(22) + 1 + max(FB(15), 0) = -1 + 1 + 0 = 0

rvore após rotação S.E. em 15:
      10[0]
     /  \
   5[0]  22[0]
  /  \    \
2[0] 8[0] 15[0] 25[0]

[PASSO 3] Removendo chave 5...
Detalhe da implementação:
1. O nó 5 possui dois filhos (2 e 8).
2. Localiza o sucessor in-order: o menor nó da subárvore direita, que é o 8.
3. Copia a chave 8 para a posição do 5.
4. Remove o nó folha original 8 da subárvore direita.
5. A subárvore direita de 8 diminuiu de altura. Recalcula FB(8):
   FB(8) = altura(esq=2) - altura(dir=vazio) = 1 - 0 = +1.
6. Como a altura total da subárvore enraizada em 8 permaneceu 2,
   a alteração de altura NÃO se propaga para a raiz 10 (FB(10) continua 0).

rvore resultante após remover 5:

```

```

      10[0]
    8[1]
2[0]      15[0]      22[0]      25[0]

=====
>> Demonstração concluída com 100% de sucesso!
```